

- ✓ If each formal parameter, local variable (if any) and return address require one word of storage then the recursion stack space needed by the compiler (which is said to implement tail recursion) for the implementation of the following algorithm is at least: *
- 2/2

Algorithm fib_t (n , curr, prev)
{
 if (n == 1)
 return prev;
 return fib_t (n - 1, curr+ prev, curr);
}

- ☐ n-1
- ☐ n+1
- ☒ c, where c is a constant
- ☐ None of the specified options
- ☐ n



- ✓ [SELECT ALL THE CORRECT OPTIONS] For algorithms specified below, 3/3
which of the following algorithm is faster [requires less time in terms of
total number of steps (steps count)]? *

Algorithm Add1(a,b,c,m,n)

```
{  
  for (i=1; i<= m; i++)  
    for (j=1; j<= n; j++)  
      c[i,j] = a[i,j] + b[i,j];  
}
```

Algorithm Add2(a,b,c,m,n)

```
{  
  for (j=1; j<= n; j++)  
    for (i=1; i<= m; i++)  
      c[i,j] = a[i,j] + b[i,j];  
}
```

☐ Add1, if $m > n$

☐ Add2, if $m < n$

☒ Add2, if $m > n$

☒ Add1, if $m < n$

☐ Other:



- ✓ [SELECT ALL THE CORRECT OPTIONS] Assuming that front and rear are initially initialized to -1, the following algorithms can be used for: *

Algorithm enqueue(item, q, n, rear)

```
{  
    // insert item, using q(0:n-1)  
    if (rear == n-1)  
        print("Overflow: the queue is full - insertion is not possible");  
    else  
    {  
        rear = rear+1;  
        q[rear] = item;  
    }  
    return rear;  
}
```

Algorithm dequeue(q, front, rear)

```
{  
    // delete an element, using q(0:n-1)  
    if (front == rear)  
        print("The queue is empty - deletion is not possible");  
    else  
    {  
        front = front - 1;  
        item = q[front];  
        print("The deleted item is", item);  
    }  
    return front;  
}
```

☐ implementing the insert and delete operations of stack

- ☐ implementing the insert and delete operations of a circular queue
- ☒ None of the specified options
- ☐ implementing the insert and delete operations of a non-circular queue



✗ If each formal parameter, local variable (if any) and return address require one word of storage then for the following algorithm the recursion stack space needed is at least: *

0/3

Algorithm myfunc(m,n)

{

If (m==0)

 return n+1;

else if(n==0)

 return myfunc(m-1, 1);

else

 return myfunc(m-1, myfunc(m,n-1));

}

- ☐ $a(2^m + 2^n) + c$, where a and c are constants
- ☐ None of these
- ☒ $am+n+c$, where a and c are constants
- ☐ $m+n$
- ☐ $am+c$, where a and c are constants



Correct answer

- ☒ $am+c$, where a and c are constants



- ✓ [SELECT ALL THE CORRECT OPTIONS] Assuming that front and rear are initially initialized to zero, the following algorithms can be used for: *

Algorithm insert (item, q, n, front, rear)

```
{
    // insert item, using q(0:n-1)
    temp = (rear+1)% n;
    if (front == rear )
        print("Overflow: the queue is full - insertion is not possible");
    else
    {
        rear = temp;
        q[rear] = item;
    }
    return rear;
}
```

Algorithm delete (q, n, front, rear)

```
{
    // delete an element, using q(0:n-1)
    // front and rear are initially initialized to zero
    if (front == rear)
        print("The queue is empty - deletion is not possible");
    else
    {
        front = (front + 1)% n ;
        item= q[front];
        print("The deleted item is", item);
    }
    return front;
}
```



- ☐ implementing the insert and delete operations of a non-circular queue
- ☐ implementing the insert and delete operations of a stack
- ☐ implementing the insert and delete operations of a circular queue
- ☒ None of the specified options



✓ Let an array be represented as $A(2:20, 3:30, 1:25; 0:25, 5:30)$. Assuming row major representation of the array (using sequential byte addressing scheme) with 2 bytes per element. If k is the address of $A(2,3,1,0,5)$ then the address of the element represented as $A(3,10,15,6,12)$ is *

4/4

- ☐ $k+3200402$
- ☒ $k+1202254$
- ☐ $k+1015612$
- ☐ $k+63$
- ☐ None of the specified options



Section B

12 of 14 points

There is no part marking. Credits would be awarded only when all the correct options are selected.



✗ *

0/2

If a polynomial is represented in the form

$$(m, e_{m-1}, b_{m-1}, e_{m-2}, b_{m-2}, \dots, e_o, b_o)$$

where,

the first entry m represents the number of nonzero terms followed by two entries for each term representing an exponent-coefficient pair.

The total number of element locations needed to store the result using the above representation when the polynomial $x^{1000} + 4x^{500} + 3x^5 + 2x^4 + 5x + 2$ is added to the polynomial $x^{500} + 4x^{200} + 9x^4 + 2x^2 + 10x + 1$ is

- ☐ 17
- ☐ 8
- ☐ None of the specified options
- ☐ 10
- ☐ 9
- ☐ 18
- ☒ 16

✗

Correct answer

- ☒ 17



- ✓ [SELECT ALL THE CORRECT OPTIONS] Assuming that front and rear are initially initialized to zero, the following algorithms can be used for: *

Algorithm insert(item, q, n, front, rear)

```
{  
    // insert item, using q(0:n-1)  
    temp = (rear+1)% n;  
    if (front == temp )  
        print("Overflow: the queue is full - insertion is not possible");  
    else  
    {  
        rear = temp;  
        q[rear] = item;  
    }  
    return rear;  
}
```

Algorithm delete(q, n, front, rear)

```
{  
    // delete an element, using q(0:n-1)  
    // front and rear are initially initialized to zero  
    if (front == rear)  
        print("The queue is empty - deletion is not possible");  
    else  
    {  
        front = (front + 1)% n ;  
        item= q[front];  
        print("The deleted item is", item);  
    }  
}
```




```
return front;
}
```

- ☐ implementing the insert and delete operations of a non-circular queue
- ☐ implementing the insert and delete operations of a stack
- ☒ implementing the insert and delete operations of a circular queue
- ☐ None of the specified options



2/2

A general polynomial $A(x)$ can be written as

$$a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

where $a_n \neq 0$ and we say that the degree of A is n .

Then one can represent $A(x)$ as an ordered list of coefficients using a one dimensional array of length $n + 2$,

$$A = (n, a_n, a_{n-1}, \dots, a_1, a_0)$$

The first element, n is the degree of A followed by the $n + 1$ coefficients in the order of decreasing exponent.

The total number of element locations needed to store the result using the above representation when the polynomial $x^{1000} + 4x^{500} + 3x^5 + 2x^4 + 5x + 2$ is added to the polynomial $x^{500} + 4x^{200} + 9x^4 + 2x^2 + 10x + 1$ is

- ☐ 1000
- ☐ 17
- ☐ None of the specified options
- ☐ 9
- ☐ 16
- ☒ 1002
- ☐ 1001





✓ A sparse matrix containing 500 elements with n non-zero elements (where $n < 500$) can be stored in a two dimensional array using *

2/2

- ☐ $n-1$ element locations
- ☐ $3n$ element locations
- ☐ None of the specified options
- ☐ $n+1$ element locations
- ☒ $3n+3$ element locations



✓ For the following, the time required in terms of total number of steps (steps count) is *

2/2

Algorithm tower(n , source, inter, dest)

```
{
    if( $n > 0$ )
    {
        tower( $n-1$ , source, dest, inter);
        print("Move disk",  $n$ , "from", source, "to", dest);
        tower( $n-1$ , inter, source, dest);
    }
}
```

- ☐ $2n+c$, where c is a constant
- ☐ $n+1$



- ☐ n
- ☒ $2^n + c$, where c is a constant



✓ [SELECT ALL THE CORRECT OPTIONS] The maximum time [in terms of total number of steps (steps count)] that can be taken by the following algorithm is *

4/4

Algorithm polyadd (a, b, c)

```
{
    p=1; q=1; r=1;
    m=a[0]; n=b[0];
    while(p<=2*m && q<=2*n)
    {
        if(a[p] == b[q])
        {
            c[r+1]=a[p+1]+b[q+1];
            if(c[r+1]!=0)
            {
                c[r]=a[p]; r=r+2;
            }
            p=p+2; q=q+2;
        }
        if(a[p]>b[q])
        {
            c[r+1]=a[p+1]; c[r]=a[p]; p=p+2; r=r+2;
        }
        if(a[p] < b[q])
        {
            c[r+1]=b[q+1]; c[r]=b[q]; q=q+2; r=r+2;
        }
    }
    while(p<=2*m)
    {
        c[r+1]=a[p+1]; c[r]=a[p]; p=p+2; r=r+2;
    }
    while(q<=2*n)
    {
        c[r+1]=b[q+1]; c[r]=b[q]; q=q+2; r=r+2;
    }
    c[0]=(r-1)/2;
}
```





- ☐ None of the specified options
- ☐ $bn+c$, when $m < n$, where b and c are constants
- ☐ $am+b$, when $m > n$, where a and b are constants
- ☒ $am+bn+c$, where a , b and c are constants



This content is neither created nor endorsed by Google. - [Terms of Service](#) - [Privacy Policy](#).

Google Forms

